
Neura OPC UA Interface

Neura Robotics GmbH

Jul 11, 2025

CONTENTS:

1	Neura OPC UA Server	1
1.1	Overview	1
1.2	Features	1
1.3	Getting Started	1
1.4	Server Configuration	2
1.5	Exposed Methods	2
1.6	Exposed Variables	2
1.7	Testing and Validation	3

NEURA OPC UA SERVER

1.1 Overview

The Neura OPC UA Server provides real-time monitoring and control capabilities for a robotic system. The server exposes variables and methods to OPC UA clients for tasks such as controlling servo movements, monitoring robot state, and interacting with digital inputs/outputs.

This server is designed to interface with any OPC UA-compatible client, enabling seamless integration and interaction with the robot.

1.2 Features

- **Power Status Monitoring:** Indicates whether the robot is powered on or off.
- **Mode Control:** Tracks and manages the robot's operating mode.
- **Collision and Fault Detection:** Provides collision and fault status of the robot.
- **Servo Commands:** Start/stop servo operations using *ServoJ* and *ServoX*.
- **Joint Angles:** Reports the current joint angles of the robot.
- **TCP Pose:** Exposes the robot's tool center point (TCP) position and orientation.
- **Digital I/O Control:** Monitors digital inputs and controls digital outputs.

1.3 Getting Started

To communicate with the server: 1. Use an OPC UA-compatible client (e.g., UaExpert, OpenOPC, or a custom client using Python, Java, etc.). 2. Connect to the server endpoint:

- Example: `opc.tcp://<server-ip>:14840/neuraopcua/server/`

3. Use the available methods and variables to interact with the robot.

1.4 Server Configuration

The server uses a YAML configuration file for customization. An example configuration is provided below:

```
opcua_config:
  endpoint: "opc.tcp://0.0.0.0:14840/neuraopcua/server/"
  namespace: "http://neurapy.robotics"
  server_name: "Neura Robotics OPC UA Server"

program_list:
1: "Program_001"
2: "Program_002"
3: "Program_003"
4: "Program_004"
5: "Program_005"
```

1.5 Exposed Methods

The following methods are available for clients to invoke:

Method Name	NodeId	Input	Output
StartServoJ	ns=2;s=Robot.StartServoJ	None	Boolean (Success)
StartServoX	ns=2;s=Robot.StartServoX	None	Boolean (Success)
StopServo	ns=2;s=Robot.StopServo	None	Boolean (Success)

1.6 Exposed Variables

The server exposes the following variables for real-time monitoring and control:

1.6.1 Robot State Variables

Variable Name	NodeId	Type	Access
PowerStatus	ns=2;s=Robot.PowerStatus	Boolean	Read-Only
ModeStatus	ns=2;s=Robot.ModeStatus	Int32	Read-Only
CollisionStatus	ns=2;s=Robot.CollisionStatus	Boolean	Read-Only
FaultStatus	ns=2;s=Robot.FaultStatus	Boolean	Read-Only
ProgramRunningStatus	ns=2;s=Robot.ProgramRunningStatus	Int32	Read-Only

1.6.2 Joint Angles

Variable Name	NodeId	Type	Access
JointAngle_1	ns=2;s=Robot.JointAngle_1	Float	Read-Only
JointAngle_2	ns=2;s=Robot.JointAngle_2	Float	Read-Only
...	...	...	...
JointAngle_N	ns=2;s=Robot.JointAngle_N	Float	Read-Only

1.6.3 TCP Pose

Variable Name	NodeId	Type	Access
TCP_X	ns=2;s=Robot.TCP_X	Float	Read-Only
TCP_Y	ns=2;s=Robot.TCP_Y	Float	Read-Only
TCP_Z	ns=2;s=Robot.TCP_Z	Float	Read-Only
TCP_A	ns=2;s=Robot.TCP_A	Float	Read-Only
TCP_B	ns=2;s=Robot.TCP_B	Float	Read-Only
TCP_C	ns=2;s=Robot.TCP_C	Float	Read-Only

1.6.4 Digital Inputs and Outputs

Variable Name	NodeId	Type	Access
DI_0	ns=2;s=Robot.DI_0	Boolean	Read-Only
DI_1	ns=2;s=Robot.DI_1	Boolean	Read-Only
...	...	...	...
DO_0	ns=2;s=Robot.DO_0	Boolean	Read/Write
DO_1	ns=2;s=Robot.DO_1	Boolean	Read/Write

1.7 Testing and Validation

- Use an OPC UA client (e.g., UaExpert) to browse the server and interact with its variables and methods.
- Ensure the server endpoint is accessible from the client.
- Verify the NodeIds and data types match the expected configuration.